

Universidad Tecnológica Nacional
Tecnatura Universitaria en Programación (TUP) — A Distancia

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Trabajo Práctico Integrador — Programación 1

Integrantes	Santiago González Gerardo Rustik	Comision: 23 Comision: 5
Materia	Programación 1	
Fecha de entrega	14/06/2026	
Repositorio	https://github.com/santyg258/gestion-paises-python	

Índice

1. Objetivo del trabajo	3
2. Marco Teórico	3
2.1 Listas	
2.2 Diccionarios	
2.3 Funciones	
2.4 Estructuras condicionales	
2.5 Ordenamientos	
2.6 Estadísticas básicas	
2.7 Archivos CSV	
3. Decisiones Técnicas y Arquitectura	6
4. Dificultades y Conclusiones	7
5. Bibliografía	8

1. Objetivo del Trabajo

El presente trabajo práctico integrador tiene como objetivo desarrollar una aplicación en Python que permita gestionar información sobre países, aplicando los conceptos aprendidos durante la cursada de Programación 1: listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema lee datos desde un archivo CSV, permite realizar consultas y genera indicadores clave a partir del dataset.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos de distintos tipos. Se define usando corchetes y los elementos se separan por comas. En este proyecto, la lista principal almacena todos los países cargados desde el CSV, donde cada elemento es un diccionario.

Ejemplo:

```
países = [{'nombre': 'Argentina', 'poblacion': 45376763, ...}, {...}]
```

Fuente: Python Software Foundation. (2024). Python Docs — Data Structures.
<https://docs.python.org/3/tutorial/datastructures.html>

2.2 Diccionarios

Un diccionario es una estructura de datos que almacena pares clave-valor. Es mutable y no ordenado por índice, sino por clave. En este proyecto, cada país se representa como un diccionario con las claves nombre, poblacion, superficie y continente.

Ejemplo:

```
pais = {'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400,  
        'continente': 'America'}
```

Fuente: Python Software Foundation. (2024). Python Docs — Dictionaries.
<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

2.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. Se define con la palabra clave def, seguida del nombre y los parámetros. En este proyecto se aplica el principio de 'una función = una responsabilidad', dividiendo el código en módulos con funciones específicas para cargar, guardar, buscar, filtrar, ordenar y calcular estadísticas.

Ejemplo:

```
def busqueda_pais(países, nom_pais):
```

Fuente: Python Software Foundation. (2024). Python Docs — Defining Functions.
<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

2.4 Estructuras Condicionales

Las estructuras condicionales permiten ejecutar bloques de código según si una condición es verdadera o falsa. Python usa if, elif y else. En este proyecto se usan para controlar el flujo del

menú, validar entradas del usuario y manejar errores como búsquedas sin resultados o rangos inválidos.

Fuente: Python Software Foundation. (2024). Python Docs — if Statements.
<https://docs.python.org/3/tutorial/controlflow.html>

2.5 Ordenamientos

Python provee la función `sorted()` que devuelve una nueva lista ordenada. Para ordenar listas de diccionarios se usa el parámetro `key` con una función `lambda` que indica el campo a comparar. El parámetro `reverse` controla si el orden es ascendente (`False`) o descendente (`True`). En este proyecto se ordenan los países por nombre, población y superficie.

Ejemplo:

```
sorted(paises, key=lambda e: e['poblacion'], reverse=False)
```

Fuente: Python Software Foundation. (2024). Python Docs — Sorting HOW TO.
<https://docs.python.org/3/howto/sorting.html>

2.6 Estadísticas Básicas

Las estadísticas básicas permiten resumir y analizar un conjunto de datos. En este proyecto se calculan: el país con mayor y menor población usando `max()` y `min()` con el parámetro `key`; el promedio de población y superficie usando `sum()` y `len()`; y la cantidad de países por continente usando un diccionario como contador.

Ejemplo de promedio:

```
promedio = sum(e['poblacion'] for e in paises) / len(paises)
```

Fuente: Python Software Foundation. (2024). Python Docs — Built-in Functions.
<https://docs.python.org/3/library/functions.html>

2.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de archivo de texto que almacena datos tabulares separados por comas. Python incluye el módulo `csv` en su biblioteca estándar, que permite leer archivos con `DictReader` (cada fila como diccionario) y escribirlos con `DictWriter`. En este proyecto el archivo `paises.csv` es la fuente de datos principal y se actualiza cada vez que el usuario agrega o modifica un país.

Ejemplo de lectura:

```
with open('paises.csv', encoding='utf-8') as archivo: lector =  
    csv.DictReader(archivo)
```

Fuente: Python Software Foundation. (2024). Python Docs — csv module.
<https://docs.python.org/3/library/csv.html>

3. Decisiones Técnicas y Arquitectura

Estructura del proyecto

Se optó por una arquitectura modular dividida en múltiples archivos .py, donde cada módulo tiene una responsabilidad única. Esto facilita la lectura, el mantenimiento y la reutilización del código.

Archivo	Responsabilidad
main.py	Menú principal y punto de entrada del programa
datos.py	Carga y guardado del archivo CSV
busquedas.py	Búsqueda de países por nombre (parcial o exacta)
filtros.py	Filtrado por continente, rango de población y superficie
ordenamiento.py	Ordenamiento por nombre, población y superficie
estadistica.py	Cálculo de estadísticas del dataset
validaciones.py	Validación de entradas del usuario

Flujo principal del programa

1. Al iniciar, main.py llama a cargar_paises() que lee el CSV y devuelve una lista de diccionarios.
2. Se muestra el menú principal en un bucle while hasta que el usuario elija salir.
3. Según la opción elegida, se llama a la función correspondiente del módulo adecuado.
4. Los resultados se muestran en consola.
5. Las operaciones de agregar y actualizar llaman a guardar_paises() para persistir los cambios en el CSV.

Decisiones de diseño destacadas

- Uso de encoding='utf-8' y newline="" en el manejo del CSV para evitar problemas con caracteres especiales (tildes, ñ) en Windows.
- Normalización de texto con unicodedata para permitir búsquedas y filtros sin importar si el usuario escribe con o sin tilde.
- Separación entre lectura (cargar_paises) y escritura (guardar_paises) para respetar el principio de responsabilidad única.
- Uso de validar_entero() y validar_texto() con bucles while para garantizar entradas válidas sin romper el programa.

4. Dificultades y Conclusiones

Dificultades encontradas

- Manejo de encoding UTF-8: Al guardar el CSV sin especificar encoding, los caracteres especiales (á, é, ó, ñ) se corrompían. Se resolvió agregando encoding='utf-8' y newline="" en ambas operaciones de apertura de archivo.
- Líneas en blanco en el CSV: En Windows, al escribir el CSV sin newline="", se generaban líneas vacías entre cada fila que causaban errores al volver a leerlo.
- Filtrado por continente con y sin tilde: El CSV almacena 'América' con tilde, pero el usuario podía escribir 'America' sin tilde. Se resolvió usando el módulo unicodedata para normalizar ambos strings antes de comparar.
- Estructura de la lista vacía en búsquedas: Inicialmente se comparaba el resultado con None en lugar de verificar si la lista estaba vacía con not lista, lo que causaba que los mensajes de 'no encontrado' nunca se mostraran.

Conclusiones

El desarrollo de este trabajo permitió afianzar el uso de las estructuras de datos fundamentales de Python: listas y diccionarios. La combinación de ambas resultó especialmente útil para representar colecciones de entidades con múltiples atributos, como los países de este proyecto.

La modularización del código en archivos separados demostró ser una práctica clave para mantener el proyecto organizado y facilitar la detección de errores. Cada módulo con una responsabilidad clara hace que el código sea más legible y fácil de mantener.

El manejo de archivos CSV introdujo conceptos importantes sobre persistencia de datos y la importancia de controlar el encoding y el formato para garantizar compatibilidad entre sistemas operativos.

5. Bibliografía / Webgrafía

1. Python Software Foundation. (2024). Python 3 Documentation — Data Structures.
<https://docs.python.org/3/tutorial/datastructures.html>
2. Python Software Foundation. (2024). Python 3 Documentation — csv module.
<https://docs.python.org/3/library/csv.html>
3. Python Software Foundation. (2024). Python 3 Documentation — Sorting HOW TO.
<https://docs.python.org/3/howto/sorting.html>
4. Python Software Foundation. (2024). Python 3 Documentation — Built-in Functions.
<https://docs.python.org/3/library/functions.html>
5. Python Software Foundation. (2024). Python 3 Documentation — unicodedata module.
<https://docs.python.org/3/library/unicodedata.html>
6. Python Software Foundation. (2024). Python 3 Documentation — Defining Functions.
<https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Links del proyecto

Repositorio GitHub: <https://github.com/santyg258/gestion-paises-python>

Video demostrativo: <https://youtu.be/y7kCam6hmQ4>

?si=Qi_5ibjqj1IW0O4y